

# C++ Programming Textbook Notes

Module: Inheritance in C++ | Types of Inheritance, Visibility Modes & Memory Architecture

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

## 1. INHERITANCE: CONCEPTUAL FOUNDATIONS & TERMINOLOGY

### What is a Class?

A **Class** is a user-defined blueprint used to model the properties (data members) and behaviors (member functions) of real-world entities into software structures.

### What is Inheritance?

**Inheritance** is a core pillar of Object-Oriented Programming (OOP). It is the process by which a new class acquires (inherits) the properties and behaviors of an existing class.

- **Existing Class:** Known as the **Base Class**, **Parent Class**, or **Super Class**.
- **New Class:** Known as the **Derived Class**, **Child Class**, or **Sub Class**.

### Why Do We Need Inheritance?

- **Code Reusability:** Instead of rewriting tested attributes and methods, the derived class inherits them directly from the base class.
- **Transitive Extensibility:** Allows developers to enhance existing libraries and domain models without modifying or recompiling original source code.
- **Hierarchical Classification:** Accurately models real-world taxonomies (e.g., Vehicle -> Car -> ElectricCar).

### How Inheritance Works in Memory

When an object of a derived class is instantiated in RAM:

1. The operating system allocates a single contiguous block of memory large enough to hold **both the inherited base class data members and the derived class's own data members**.
2. In the memory block, the base class sub-object is laid out first, followed immediately by the derived class sub-object.
3. Even if base class members are `private`, they still physically occupy space inside the derived object's RAM allocation—they are simply hidden by compiler access rules!

[ DERIVED OBJECT MEMORY ALLOCATION IN RAM ]

```
Base Class:    class A { int a; };           // 4 Bytes
Derived Class: class B : public A { int b; }; // Inherits A + adds 4 Bytes
```

Instantiation: B obj; (Allocates contiguous 8 Bytes in RAM at 0x7ffd00)

|                                        |                                           |
|----------------------------------------|-------------------------------------------|
| Base Sub-Object (A)<br>int a (4 Bytes) | Derived Sub-Object (B)<br>int b (4 Bytes) |
| Address: 0x7ffd00                      | Address: 0x7ffd04                         |

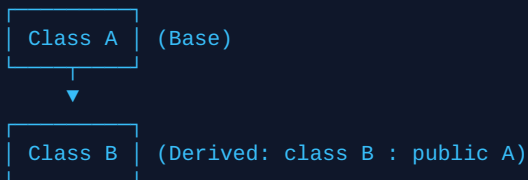
## 2. SYNTAX & THE 5 TYPES OF INHERITANCE

### General Inheritance Syntax

```
class BaseClass {  
    // Base class members  
};  
  
class DerivedClass : visibility_mode BaseClass {  
    // Derived class members  
};
```

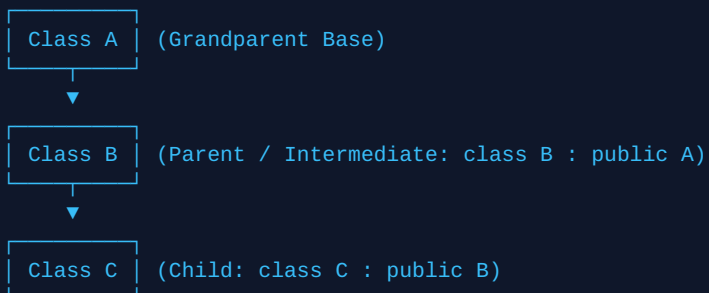
### 1. Single Inheritance

A derived class inherits directly from a **single base class**.



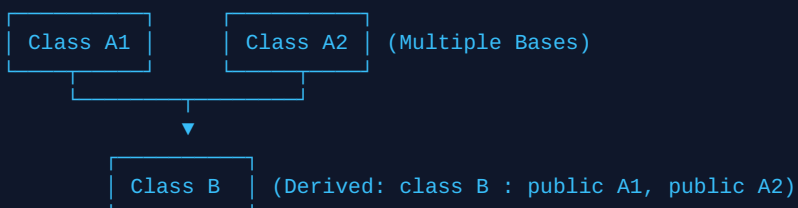
### 2. Multilevel Inheritance

A class is derived from another derived class, forming an inheritance chain.



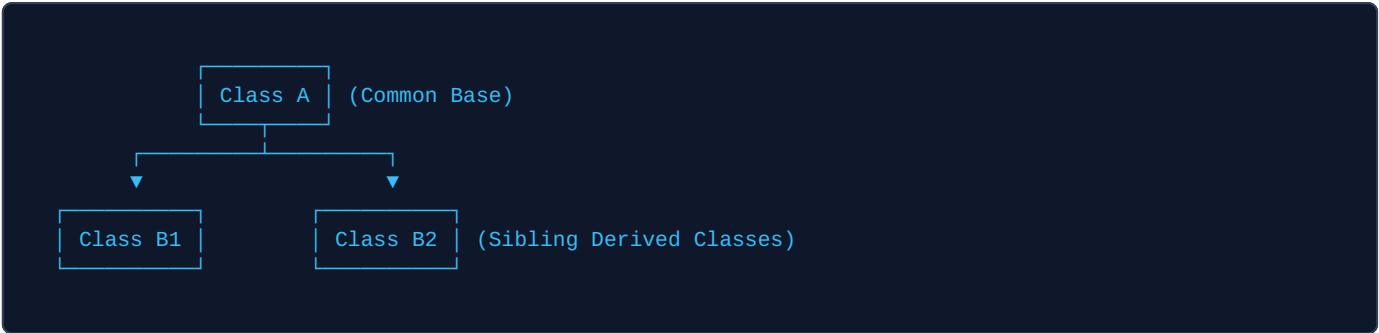
### 3. Multiple Inheritance

A single derived class inherits directly from **two or more base classes** simultaneously.



### 4. Hierarchical Inheritance

Multiple derived classes inherit from a **single common base class**.



### 5. Hybrid Inheritance

A combination of two or more distinct types of inheritance (e.g., Multilevel + Multiple), often forming a Diamond hierarchy.

## 3. VISIBILITY MODES & ACCESS CONTROL MECHANICS

### The Concept of "Types of Users"

To understand access specifiers and visibility modes deeply, consider the two distinct consumers of any class:

- 1. **User 1 (The Client / Object Instantiator):** Creates objects of your class in functions like `main()`. User 1 can **only access public members**.
- 2. **User 2 (The Class Extender / Derived Class Author):** Derives a new child class from your class. User 2 can access both **public and protected members**, but *never* private members!

**What is the 'protected' Access Specifier?**

A member declared as protected is inaccessible to outside objects (User 1), but accessible to derived child classes (User 2). It provides encapsulation with inheritance flexibility.

### The 3 Visibility Modes: Controlling Inheritance Inheritance Impact

The **Visibility Mode** specifies how inherited base members are categorized inside the derived class. Its impact is strictly on the **users of the derived class (B)**:

| Base Member Access | public Inheritance           | protected Inheritance        | private Inheritance          |
|--------------------|------------------------------|------------------------------|------------------------------|
| private            | Not Inherited (Inaccessible) | Not Inherited (Inaccessible) | Not Inherited (Inaccessible) |
| protected          | Remains protected in B       | Becomes protected in B       | Becomes private in B         |
| public             | Remains public in B          | Becomes protected in B       | Becomes private in B         |

**Crucial Takeaway on Visibility Modes**

Base class private members are **never accessible** in the derived class directly under any visibility mode. If class B inherits class A privately, protected and public members of A become private inside B, meaning subclasses of B (User 2 of B) cannot access them!

## 4. PRACTICAL CODE DEMONSTRATIONS

### Example 1: Protected Access & Single Inheritance

```
// Demonstrating Protected Access Specifiers and Single Inheritance
#include <iostream>
using namespace std;

class A {
private:
    int a; // Inaccessible to derived classes

protected:
    // Accessible to derived class B (User 2), hidden from main() (User 1)
    void setValue(int k) {
        a = k;
    }

public:
    void displayA() {
        cout << "Value of a (Base A): " << a << endl;
    }
};

// Class B inherits publicly from Class A
class B : public A {
public:
    void setData(int x) {
        // Direct call to protected member function of Base Class A
        setValue(x);
    }
};

int main() {
    B obj;
    // obj.setValue(10); // ERROR: setValue is protected in A!
    obj.setData(4);      // Valid: Public method of B
    obj.displayA();      // Valid: Inherited public method of A
    return 0;
}
```

## Example 2: Multilevel and Multiple Inheritance Demonstrations

```
// Multilevel and Multiple Inheritance Syntactic Implementation
#include <iostream>
using namespace std;

// Multiple Inheritance Bases
class Engine {
public:
    void startEngine() { cout << "Engine Started!" << endl; }
};

class Radio {
public:
    void playMusic() { cout << "Playing Music!" << endl; }
};

// Multiple Inheritance: Car inherits from both Engine and Radio
class Car : public Engine, public Radio {
public:
    void drive() { cout << "Car is Driving..." << endl; }
};

int main() {
    Car myCar;
    myCar.startEngine(); // From Engine base
    myCar.playMusic();   // From Radio base
    myCar.drive();       // From Car derived
    return 0;
}
```

### Best Practice: Default Visibility Mode in C++

If you omit the visibility mode in a class declaration (e.g., `class B : A`), C++ defaults to **private inheritance**! Always explicitly write `public` (e.g., `class B : public A`) when modeling "is-a" relationships.